

Credit default prediction model

Methodology demonstration

Roman Kochnev

May 31, 2026

1 Research question & EDA

The purpose of this assignment is to build a risk prediction based on the dataset from the book 'Credit Scoring and Its Applications' by Thomas et al. 2002 (TEC-2002-LOAN-DATA.csv). Exploratory Data Analysis, model estimation and model evaluation will be done using R. The dataset contains some amount of observations, the variable of interest ('bad'=0 or 1, hereafter *response*) and the variables that may influence the variable of interest (hereafter *predictors*).

Before we start, we need to load the data, load the relevant libraries, define the working directory and the random seed (for the results replication). The library 'tidyverse' is needed for data manipulations, the library 'pROC' is needed for comparative model evaluation.

```
> library(tidyverse)
> library(pROC)
> set.seed(1337)
> setwd("D:/your/folder/with/the_data")
> df = read.csv("TEC-2002-LOAN-DATA.csv", header = TRUE)
```

Before we start building models, we should check the provided dataset first: we need to find out, whether the overall data structure corresponds to that stated

in the manual, whether there are any NaN-values, blanks, missing values, wrong entries, outliers, to find out how balanced or unbalanced is the dataset, to get the amount of observations overall and potentially compromised observations. The code that covers those tasks lies in the appendix inside sections labeled as 'DATA EXPLORATION', 'ERROR CHECK' and 'OUTLIER CONTROL'.

The provided dataset contains 1225 observations, and it does not contain compromised values: no wrong entries or typos related to input error, no NaNs, no blanks, no even zero balance values ('000001'). The only thing we have here are 7 observations with the year of birth = 99, which is equal to 'unknown'. Those 7 observations can be easily eliminated from the dataset.

```
> #Remove 7 records with unknown year of birth (yob == 99)
> df = df[-which(df$yob == 99), ]
> #Overview
> cat("n =", nrow(df), "| bad rate =", round(mean(df$bad), 4), "\n")
```

```
n = 1218 | bad rate = 0.2627
```

The dataset is definitely unbalanced: the number of 'good' applicants ($bad=0$) is almost three times higher than the number of 'bad' ($bad=1$) ones. Since the rate of 'bad' applicants $\approx 26,37\%$, the straightforward application of a model may give incorrect results, in particular: it will decrease the true positive rate (also known as *sensitivity*). It means, if we do not take the imbalance into account, the applied model will unreasonably mark truly 'bad' applicants as 'good', because it will be biased towards 'good' status. In order to demonstrate this, we will build two competitive models: a model based on unbalanced dataset (as it is now) and a model based on balanced dataset (where the amount of applicants with 'bad'=0 is the same as of those with 'bad'=1).

The distribution of numeric values seems plausible. All the predictors are heavily right-skewed and have very long tails. The only exceptions are Value of Home, Mortgage balance, Outgoings on mortgage/rent (not so heavily skewed, lesser kurtosis) and Applicant's income (same plus double peaks), see Figure 1. In order to inspect the situation related to the tails and possible outliers of the remaining

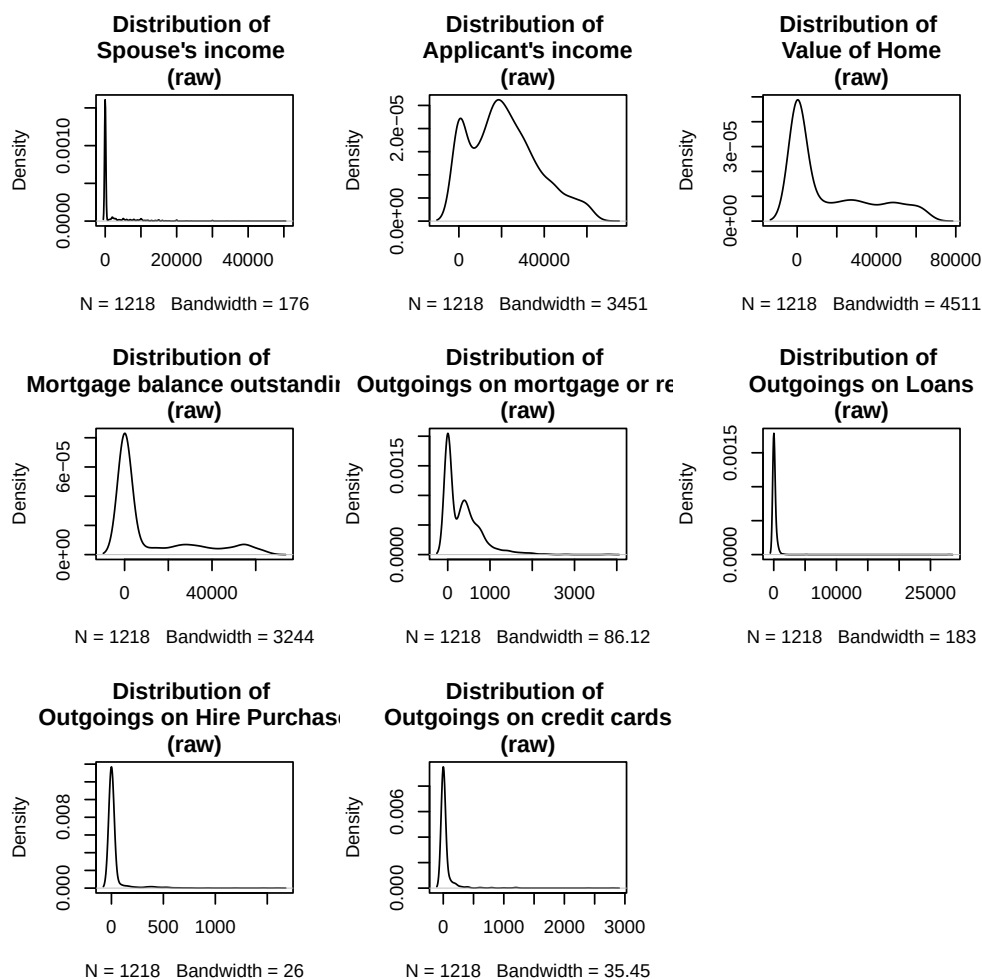


Figure 1: Distribution of the numeric, non-categorical data (raw)

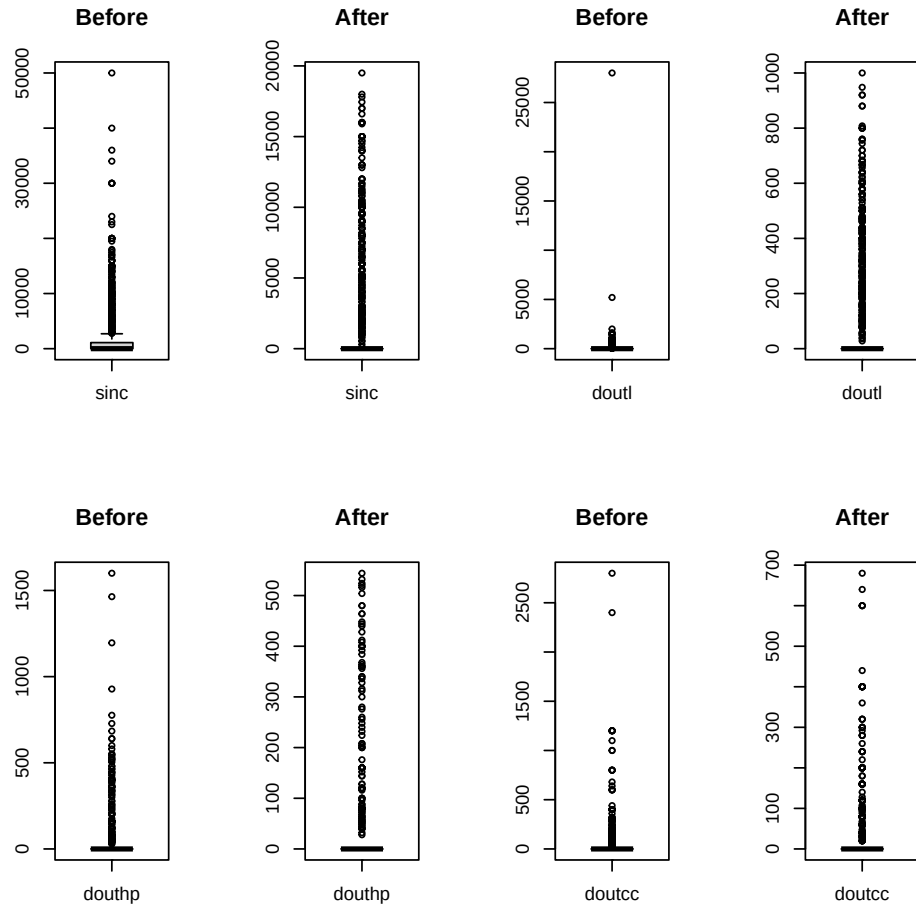


Figure 2: Distribution of numeric variables before and after removal of the outliers

numeric variables we can apply boxplots, and if we do, we would see very long trails of bread crumbs with a few quite distant values (see Figure 2, columns 'Before'). So the dataset does contain observations with exceptional distant values. Actually, those values look plausible in terms of economic environment, so in fact one *could* keep them in, but since linear models (and also generalized linear models we are going to use) are sensitive to outliers, it would be better to remove them from the dataset. In this particular case, we remove the observations that do not belong to the 99th percentile: thus we omit 54 observations.

Additionally we will slightly transform the variable 'yob' (year of birth): we turn it into an age, which is a more intuitive value. So for instance instead of year of birth = 72 (i.e. 1972) we will get the age at the time of 2002 (which would be 30). So we turn it into age and the centre.

With this refined dataset we begin definition of two competing models.

2 Generalized linear models

An industrial standard of predicting risks like that (where a response is a boolean variable) is generalized linear model, to be more precise - a logit model. One could use a probit model here and that would not be a mistake, but the output of a logit model is more intuitive in interpretations due to natural interpretations in terms of odds ratio.

First we split the data on two sets: train data and test data in an 80/20 ratio. The split implies that 80% of 'good' applicants ($bad=0$) lie in the train dataset as well as 80% of the 'bad' applicants ($bad=1$) were placed in the same set, thus 80% of 'bad' applicants are in the train set and 20% of 'bad' applicants are in the 'test' set, same for the 'good' applicants. That gives us an initial train dataset with 936 observations and an initial test dataset with 235 observations. Both sets have a 'bad' rate=0,2714 (it has increased due to removal of the outliers).

2.1 Imbalanced GLM

After we have splitted the data, we can start fitting a logit model. First we declare a full model, i.e. a model with all predictors, and then we take a look at all the predictors with $Pr(> |z|)$ below 0,2. Technically one could start not with $< 0,2$ but with $\leq 0,05$, but to be on the safe side, one can take a less conservative value: you never know how the values will change when you eliminate a large number of predictors.

Call:

```
glm(formula = bad ~ ., family = binomial, data = df_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	-3.475e-01	6.056e-01	-0.574	0.56611
yob	-1.620e-02	8.630e-03	-1.877	0.06054 .
nkid1	3.422e-02	2.710e-01	0.126	0.89953
nkid2	2.960e-02	2.588e-01	0.114	0.90892
nkid3	2.622e-02	4.049e-01	0.065	0.94837
nkid4	-9.162e-01	7.922e-01	-1.157	0.24745
nkid5	-1.353e+01	4.653e+02	-0.029	0.97680
dep	-1.917e-02	4.084e-01	-0.047	0.96257
phon1	-3.397e-01	2.626e-01	-1.294	0.19580
sinc	-1.039e-05	2.487e-05	-0.418	0.67598
aesE	-2.089e-01	5.562e-01	-0.376	0.70725
aesM	-1.720e-01	7.599e-01	-0.226	0.82093
aesN	-5.301e-01	1.298e+00	-0.409	0.68287
aesP	-2.963e-01	5.135e-01	-0.577	0.56397
aesR	-9.177e-02	6.318e-01	-0.145	0.88451
aesT	-1.254e-01	5.761e-01	-0.218	0.82770
aesU	-3.481e-02	1.073e+00	-0.032	0.97412
aesV	-7.133e-01	5.392e-01	-1.323	0.18586
aesW	1.494e-01	6.985e-01	0.214	0.83063

aesZ	-2.706e-01	1.186e+00	-0.228	0.81954
dainc	-2.811e-05	6.880e-06	-4.086	4.39e-05 ***
resN	1.192e+00	4.127e-01	2.889	0.00387 **
resO	4.549e-01	3.695e-01	1.231	0.21822
resP	4.434e-01	3.156e-01	1.405	0.16006
resU	1.930e-01	3.695e-01	0.522	0.60153
dhval	-2.512e-06	5.554e-06	-0.452	0.65102
dmort	2.498e-06	5.757e-06	0.434	0.66441
doutm	3.780e-04	2.447e-04	1.545	0.12230
doutl	9.209e-05	4.843e-04	0.190	0.84918
douthp	6.085e-05	1.078e-03	0.056	0.95497
doutcc	-1.134e-03	1.287e-03	-0.881	0.37814

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1087.9 on 931 degrees of freedom
 Residual deviance: 1013.4 on 901 degrees of freedom
 AIC: 1075.4

Number of Fisher Scoring iterations: 13

	Estimate	Std. Error	z value	Pr(> z)
yob	-0.0162	0.0086	-1.8769	0.0605
phon1	-0.3397	0.2626	-1.2936	0.1958
aesV	-0.7133	0.5392	-1.3229	0.1859
dainc	0.0000	0.0000	-4.0859	0.0000
resN	1.1922	0.4127	2.8889	0.0039
resP	0.4434	0.3156	1.4049	0.1601
doutm	0.0004	0.0002	1.5452	0.1223

Then we build a reduced model (let's call it 'logit2') that contains only the predictors with $Pr(> |z|) < 0,2$.

Call:

```
glm(formula = bad ~ yob + phon + aes + dainc + res + doutm, family = binomial,  
     data = df_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-3.248e-01	5.975e-01	-0.544	0.58668	
yob	-1.581e-02	8.560e-03	-1.847	0.06480	.
phon1	-3.647e-01	2.603e-01	-1.401	0.16113	
aesE	-2.621e-01	5.453e-01	-0.481	0.63076	
aesM	-1.970e-01	7.417e-01	-0.266	0.79054	
aesN	-6.938e-01	1.256e+00	-0.552	0.58069	
aesP	-3.279e-01	5.027e-01	-0.652	0.51419	
aesR	-9.945e-02	6.108e-01	-0.163	0.87066	
aesT	-1.535e-01	5.654e-01	-0.272	0.78598	
aesU	-1.004e-01	1.063e+00	-0.095	0.92468	
aesV	-7.495e-01	5.288e-01	-1.417	0.15639	
aesW	9.171e-02	6.784e-01	0.135	0.89246	
aesZ	-2.464e-01	1.171e+00	-0.210	0.83338	
dainc	-2.795e-05	6.653e-06	-4.201	2.66e-05	***
resN	1.210e+00	4.085e-01	2.962	0.00306	**
resO	3.970e-01	3.124e-01	1.271	0.20378	
resP	4.592e-01	3.134e-01	1.465	0.14287	
resU	1.820e-01	3.577e-01	0.509	0.61086	
doutm	3.981e-04	2.241e-04	1.776	0.07571	.

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1087.9 on 931 degrees of freedom
Residual deviance: 1018.6 on 913 degrees of freedom

AIC: 1056.6

Number of Fisher Scoring iterations: 4

	Estimate	Std. Error	z value	Pr(> z)
yob	-0.0158	0.0086	-1.8466	0.0648
dainc	0.0000	0.0000	-4.2007	0.0000
resN	1.2098	0.4085	2.9617	0.0031
doutm	0.0004	0.0002	1.7761	0.0757

It seems that *aes* is not very significant as well as *phon*. *yob* seems insignificant as well. So we remove *aes* and *phone* and define a third logit model ('logit3').

Call:

```
glm(formula = bad ~ yob + dainc + res + doutm, family = binomial,  
     data = df_train)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	-7.827e-01	2.624e-01	-2.983	0.00286	**
yob	-1.825e-02	5.978e-03	-3.053	0.00227	**
dainc	-3.348e-05	5.830e-06	-5.743	9.32e-09	***
resN	1.141e+00	3.985e-01	2.864	0.00419	**
res0	2.934e-01	2.945e-01	0.996	0.31908	
resP	3.881e-01	3.038e-01	1.277	0.20149	
resU	7.476e-02	3.418e-01	0.219	0.82685	
doutm	3.896e-04	2.225e-04	1.751	0.07994	.

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 1087.9 on 931 degrees of freedom

Residual deviance: 1027.0 on 924 degrees of freedom
AIC: 1043

Number of Fisher Scoring iterations: 4

Thus we've obtained a reasonably reduced model: all predictors are significant, the amount of predictors is low, the reduction has significantly decreased AIC while the increase of residual deviance is minor. The tests have shown that introduction of interactions between any predictors did not improve the model. It is worth mentioning that the predictors configuration would be different wouldn't we have eliminated the observations with *yob* = 99.

Now we can use the predictors configuration in the final reduced model as a starting point for the model based on the balanced dataset.

2.2 Balanced GLM

Strategy: for each of $B = 500$ iterations, draw a *balanced* bootstrap sample (equal n from *bad*=1 and *bad*=0 with replacement), fit the same logit specification, and collect the predicted probs on the held-out test set. Final prediction = average across B .

This forces each sub-model to learn from a 50/50 dataset, preventing the majority-class bias that causes the plain logit to almost never predict a default at threshold equal to 0,5.

2.3 Evaluation and conclusion

In order to compare and evaluate the competitive models, we will use the following metrics and techniques:

- Confusion matrix (true positives, true negatives, false positives, false negatives), accuracy, sensitivity and specificity. The *sensitivity* (also known as *true positive rate*) is the matter of special interest, since we expect the imbalanced

dataset to force a model to become biased towards prediction $bad = 0$. Higher - better.

- F1 score - the harmonic mean of precision and recall. Higher - better, the perfect score is 1.
- Brier score - the average squared difference between predicted probabilities and actual outcomes. It is invariant to a threshold. Lower - better, the perfect score is 0.
- Youden's J statistic - the probability cutoff in a binary classification model that maximizes the model's ability to distinguish between positive and negative classes. It is calculated by finding the threshold that maximizes the difference between the True Positive Rate and False Positive. Youden-optimal threshold ensures that the model performs reasonably well at catching the minority class while keeping high specificity.
- Area under curve (AUC) - an area under the line where asymptote axis is FPR (false positive rate) and ordinate axis is TPR (true positive rate). Higher - better. A common evaluation technique for unbalanced datasets and for tasks with relatively high cost of false positives.

Brier score: *mylogit3* is better (0,1872 vs 0,2295; see Table 1 or Table 2 in Appendix) - its probabilities are closer on average to 0/1, but only *because they are pulled toward the majority-class base rate*, not because the model is better calibrated for credit risk decisions.

At *Youden-optimal* threshold, the two models are *nearly equivalent* in all metrics, with the *bootstrap ensemble* having a *slight edge* in AUC, accuracy, specificity, and F1 (see Table 2).

At threshold 0,5 (the practical default):

- *mylogit3* catches less than 10% of actual defaults (Sens=0,09).
- Bootstrap ensemble catches more than 50% (Sens=0,508).

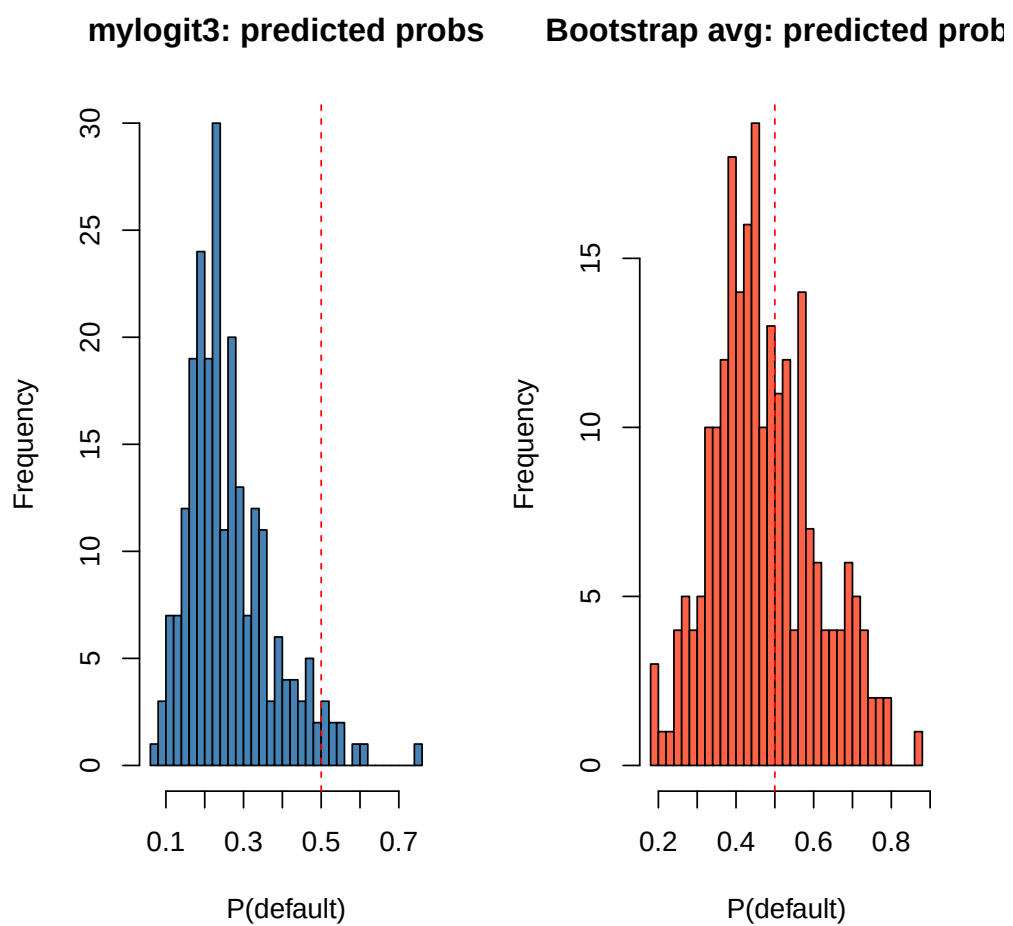


Figure 3: Predicted probability histograms

===== FINAL SUMMARY (Youden-optimal) =====

Metric	mylogit3	Bootstrap	Better

acc	0.7382	0.7425	= Boot
sens	0.3333	0.3333	Tie
spec	0.8882	0.8941	= Boot
f1	0.4078	0.4118	= Boot
brier	0.1872	0.2295	logit3 ->
AUC	0.6285	0.6328	= Boot

Table 1: Short table of comparative characteristics of two models

This is the most important difference: without threshold tuning, the unbalanced model is nearly useless for identifying defaulters.

AUC: Bootstrap ensemble is marginally *better* (0,6328 vs 0,6285; see Table 1 or Figure 4 in Appendix).

The *bootstrap* ensemble is the better model for credit default prediction: it is more robust (no threshold tuning needed), achieves higher AUC, and correctly trades off sensitivity/specificity across all threshold choices (for further details see Table 1 and Table 2).

3 Appendix

3.1

```
library(tidyverse)
library(pROC)          # roc(), auc(), coords()
set.seed(1337)

setwd("E:/JKU Study 2/2026S/Banking")

df = read.csv("TEC-2002-LOAN-DATA.csv", header = TRUE)
# DATA EXPLORATION
#Overall structure
str(df)
summary(df)

# ERROR CHECK START
## how many people with unknown Year of birth?
print(sum(df['yob']==99))
print(nrow(df))
## 7 out of 1225
## Any NA in the dataset?
for (i in df) {
  print(sum(is.na(i)))
}
## Any false entries?
aes_check = c('V',
              'W',
              'M',
              'P',
              'B',
              'R',
```

```

        'E',
        'T',
        'U',
        'N',
        'Z')
aes_check_count = 0
for (i in unique(df['aes'])) {
  aes_check_count = aes_check_count + i%in% aes_check
}
aes_check_count = sum(aes_check_count)
aes_check_count;nrow(unique(df['aes']))
### Match!

res_check = c('O',
              'F',
              'U',
              'P',
              'N',
              'Z')
res_check_count = 0
for (i in unique(df['res'])) {
  res_check_count = res_check_count + i%in% res_check
}
res_check_count = sum(res_check_count)
res_check_count;nrow(unique(df['res']))
### Match!

## 000001 anywhere?
sort(unique(df$dhval))
sort(unique(df$dmort))

sort(unique(df$doutm))

```

```

sort(unique(df$doutl))
sort(unique(df$douthp))
sort(unique(df$doutcc))

sum(sort(unique(df$dhval)) >0 & sort(unique(df$dhval)) <1)
sum(sort(unique(df$dmort)) >0 & sort(unique(df$dmort)) <1)
sum(sort(unique(df$doutm)) >0 & sort(unique(df$doutm)) <1)
sum(sort(unique(df$doutl)) >0 & sort(unique(df$doutl)) <1)
sum(sort(unique(df$douthp))>0 & sort(unique(df$douthp))<1)
sum(sort(unique(df$doutcc))>0 & sort(unique(df$doutcc))<1)
# No 000001 anywhere or anything similar

## No NA, errors
#   ERROR CHECK END

#Remove 7 records with unknown year of birth (yob == 99)
df = df[-which(df$yob == 99), ]

#Overview
cat("n =", nrow(df), "| bad rate =", round(mean(df$bad), 4), "\n")
#bad=0: 898, bad=1: 320
#heavily unbalanced (~26% default rate)

#   DATA PREPARATION
df$yob = df$yob + 1900-2002 # turn year of birth into age
df$yob = df$yob - mean(df$yob) # centre age
df$phon = factor(df$phon)
df$nkid = factor(df$nkid)

#   OUTLIER CONTROL
## Distribution of numeric vars
num_var_names = c(

```



```

'Spouse\'s income',
'Applicant\'s income',
'Value of Home',
'Mortgage balance outstanding',
'Outgoings on mortgage or rent',
'Outgoings on Loans',
'Outgoings on Hire Purchase',
'Outgoings on credit cards'
)
par(mfrow = c(3, 3))
plot(density(df$sinc), main="Distribution of
\nSpouse's income (raw)")
plot(density(df$dainc), main="Distribution of
\nApplicant's income (raw)")
plot(density(df$dhval), main="Distribution of
\nValue of Home (raw)")
plot(density(df$dmort), main="Distribution of
\nMortgage balance outstanding (raw)")
plot(density(df$doutm), main="Distribution of
\nOutgoings on mortgage or rent (raw)")
plot(density(df$doutl), main="Distribution of
\nOutgoings on Loans (raw)")
plot(density(df$douthp), main="Distribution of
\nOutgoings on Hire Purchase (raw)")
plot(density(df$doutcc), main="Distribution of
\nOutgoings on credit cards (raw)")
par(mfrow = c(1, 1))

df_numerics = cbind(
  df$sinc,
  df$dainc,
  df$dhval,

```

```

df$dmort,
df$doutm,
df$doutl,
df$douthp,
df$doutcc)
colnames(df_numerics) = c(
  'sinc',
  'dainc',
  'dhval',
  'dmort',
  'doutm',
  'doutl',
  'douthp',
  'doutcc'
)
df_numerics = as.data.frame(df_numerics)
probs = c(0.25,0.5,0.75,0.99,1)

quantile_table=matrix(data = NA, nrow = length(df_numerics),
ncol = length(probs))
rownames(quantile_table) = colnames(df_numerics)
colnames(quantile_table)=1:5

for (i in 1:length(probs)) {
  colnames(quantile_table)[i] = paste0(probs[i]*100,'%')
}

for (i in 1:length(df_numerics)) {
  quantile_table[i,] = quantile(df_numerics[,i], probs = probs)
}
par(mfrow = c(3, 3))
for (i in 1:length(df_numerics)) {

```

```

    boxplot(df_numerics[i], show.names=TRUE, main=num_var_names[i])
}
par(mfrow = c(1, 1))
rm(df_numerics, probs)
## Asymmetric, very long tails
## Variables dainc, dhval, dmort have no strong outliers
## dmort also acceptable

## Create a dataset that does not contain terrible outliers
num_vars_intact = c(
  'dainc',
  'dhval',
  'dmort',
  'doutm'
)

cutoff_percentile = subset(quantile_table,
!rownames(quantile_table) %in% num_vars_intact)
cutoff_percentile = cutoff_percentile[,4]

df_pretrain = df

## Cut-off all the observations that do not belong to 99% percentile
for (i in labels(cutoff_percentile)) {
  cat("Items to remove at", i, ": ")
  eval(parse(text=(paste0("print(sum(df_pretrain$", i, ">=",
as.numeric(cutoff_percentile[i]), ")),", sep="")))))
  eval(parse(text=(
paste0("df_pretrain = df_pretrain[df_pretrain$", i,
"<", as.numeric(cutoff_percentile[i]), "],", sep="")))))
}
nrow(df)-nrow(df_pretrain)

```

```

par(mfrow = c(2, 4))
for (i in labels(cutoff_percentile)) {
  boxplot(df[i], show.names=TRUE, main=labels(cutoff_percentile)[i])
  title(main = "Before")
  boxplot(df_pretrain[i], show.names=TRUE, main=labels(cutoff_percentile)[i])
  title(main = "After")
}
par(mfrow = c(2, 4))
for (i in labels(cutoff_percentile)) {
  eval(parse(text=(paste0("plot(density(df$",i,"),main = 'Before')",sep="")))))
  eval(parse(text=(paste0("plot(density(df_pretrain$",i,"),
    main = 'After')",sep="")))))
}
par(mfrow = c(1, 1))

df = df_pretrain

# STRATIFIED TRAIN / TEST SPLIT 80 / 20
# Stratify by 'bad' so both splits have the same default rate.
idx_bad1 = which(df$bad == 1)
idx_bad0 = which(df$bad == 0)

train_idx = c(
  sample(idx_bad1, size = floor(0.80 * length(idx_bad1)), replace = FALSE),
  sample(idx_bad0, size = floor(0.80 * length(idx_bad0)), replace = FALSE)
)
df_train = df[ train_idx, ]
df_test  = df[-train_idx, ]

cat(sprintf("Train n=%d | bad rate=%.4f\n",
  nrow(df_train), mean(df_train$bad)))

```

```

cat(sprintf("Test  n=%d | bad rate=%.4f\n",
nrow(df_test),  mean(df_test$bad)))

# =====
# PART 1 - full, UNBALANCED training data
# =====
## Logit (all variables)
mylogit = glm(bad ~ .,
              family = binomial,
              data = df_train)
summary(mylogit)
### select variables with Pr(>|z|) below 0.2
mylogit_varnames_filter = summary(mylogit)$coefficients
mylogit_varnames_filter = subset(mylogit_varnames_filter,
mylogit_varnames_filter[,4]<=0.2)
print(round(mylogit_varnames_filter,4))

## Logit 2 (first iteration)
mylogit2 = glm(bad ~ yob + phon + aes + dainc + res + doutm,
              family = binomial,
              data = df_train)
summary(mylogit2)
### select variables with Pr(>|z|) below 0.1
mylogit_varnames_filter = summary(mylogit2)$coefficients
mylogit_varnames_filter = subset(mylogit_varnames_filter,
mylogit_varnames_filter[,4]<=0.1)
print(round(mylogit_varnames_filter,4))

# Variables retained from stepwise filtering (Pr < 0.1):
# yob, sinc, dainc, res, doutcc

```

```

mylogit3 = glm(bad ~ yob + dainc + res + doutm,
               family = binomial,
               data    = df_train)
summary(mylogit3)

# Predict on test set (type = "response" => probabilities)
pred_logit3 = predict(mylogit3, newdata = df_test, type = "response")

# =====
# BOOTSTRAPPING - balanced logit ensemble
# =====
# Strategy: for each of B iterations, draw a BALANCED bootstrap
# sample (equal n from bad=1 and bad=0 with replacement), fit
# the same logit specification, and collect the predicted probs
# on the held-out test set. Final prediction = average across B.
#
# This forces each sub-model to learn from a 50/50 dataset,
# preventing the majority-class bias that causes the plain
# logit to almost never predict a default at threshold = 0.5.
#

B = 500 # number of bootstrap models

train_bad1 = df_train[df_train$bad == 1, ]
train_bad0 = df_train[df_train$bad == 0, ]
n_min      = nrow(train_bad1) # minority class size in train

cat(sprintf("\nBootstrap setup: B=%d | n_min=%d | n_maj=%d\n",
           B, n_min, nrow(train_bad0)))

# Matrix to collect one column of test-set predictions per model

```

```

boot_pred_matrix = matrix(NA_real_, nrow = nrow(df_test), ncol = B)

for (b in seq_len(B)) {
  # Balanced resample: n_min rows from each class, with replacement
  idx1 = sample(nrow(train_bad1), size = n_min, replace = TRUE)
  idx0 = sample(nrow(train_bad0), size = n_min, replace = TRUE)
  df_boot = rbind(train_bad1[idx1, ], train_bad0[idx0, ])

  # Same formula as mylogit3
  boot_fit = glm(bad ~ yob + dainc + res + doutm,
                 family = binomial,
                 data = df_boot,
                 control = list(maxit = 100))

  boot_pred_matrix[, b] = predict(boot_fit,
                                   newdata = df_test,
                                   type = "response")
}

# Ensemble prediction: average the 500 probability columns
pred_boot_avg = rowMeans(boot_pred_matrix, na.rm = TRUE)

# =====
# MODEL COMPARISON - test-set evaluation
# =====

# Define function to return confusion matrix data,
# accuracy/sensitivity/specificity, F1, Brier
eval_model = function(actual, probs, threshold, label) {
  pred_class = as.integer(probs >= threshold)
  cm = table(factor(actual, levels = 0:1),

```

```

        factor(pred_class, levels = 0:1))
TN    = cm[1, 1]; FP = cm[1, 2]
FN    = cm[2, 1]; TP = cm[2, 2]
acc   = (TP + TN) / sum(cm)
sens  = TP / (TP + FN)
spec  = TN / (TN + FP)
prec  = ifelse((TP + FP) > 0, TP / (TP + FP), 0)
f1    = ifelse((prec + sens) > 0, 2 * prec * sens / (prec + sens), 0)
brier = mean((actual - probs)^2)
cat(sprintf(
  "  %-42s  Acc=%.3f  Sens=%.3f  Spec=%.3f  F1=%.3f  Brier=%.4f\n",
  label, acc, sens, spec, f1, brier))
cat(sprintf("    Confusion: TN=%d  FP=%d  FN=%d  TP=%d\n",
  TN, FP, FN, TP))
invisible(list(acc=acc, sens=sens, spec=spec, f1=f1, brier=brier,
  TN=TN, FP=FP, FN=FN, TP=TP))
}

# ROC / AUC
roc_logit3 = roc(df_test$bad, pred_logit3, quiet = TRUE)
roc_boot   = roc(df_test$bad, pred_boot_avg, quiet = TRUE)

cat(sprintf("\n%-42s  AUC = %.4f\n", "mylogit3",          auc(roc_logit3)))
cat(sprintf("%-42s  AUC = %.4f\n", "Bootstrap ensemble", auc(roc_boot)))

# Youden-optimal threshold
# Maximizes sensitivity + specificity - 1
thresh_l3   = coords(roc_logit3, "best",
  best.method = "youden", ret = "threshold")[[1]]
thresh_boot = coords(roc_boot, "best",
  best.method = "youden", ret = "threshold")[[1]]

```



```

cat(sprintf("\nYouden thresholds: mylogit3=%.3f  bootstrap=%.3f\n",
           thresh_l3, thresh_boot))

# Fixed threshold = 0.5
cat("\n===== Fixed threshold = 0.5 =====\n")
r1 = eval_model(df_test$bad, pred_logit3, 0.5, "mylogit3 (t=0.50)")
r2 = eval_model(df_test$bad, pred_boot_avg, 0.5, "Bootstrap (t=0.50)")

# Youden-optimal thresholds
cat(sprintf(
"\n===== Youden-optimal threshold (l3=%.3f, boot=%.3f) =====\n",
           thresh_l3, thresh_boot))
r3 = eval_model(df_test$bad, pred_logit3,
               thresh_l3, sprintf("mylogit3 (t=%.3f)", thresh_l3))
r4 = eval_model(df_test$bad, pred_boot_avg,
               thresh_boot, sprintf("Bootstrap (t=%.3f)", thresh_boot))

# Summary table
cat("\n===== FINAL SUMMARY (Youden-optimal) =====\n")
cat(sprintf(" %-10s %-10s %-10s %-10s\n",
           "Metric", "mylogit3", "Bootstrap", "Better"))
cat(strrep("-", 50), "\n")

for (k in c("acc", "sens", "spec", "f1", "brier")) {
  v3 = r3[[k]]; vb = r4[[k]]
  better = if (k == "brier") {
    if (vb < v3) "= Boot" else if (v3 < vb) "logit3 ->" else "Tie"
  } else {
    if (vb > v3) "= Boot" else if (v3 > vb) "logit3 ->" else "Tie"
  }
  cat(sprintf(" %-10s %-10.4f %-10.4f %-10s\n", k, v3, vb, better))
}

```

```

cat(sprintf("  %-10s  %10.4f  %10.4f  %10s\n",
            "AUC",
            as.numeric(auc(roc_logit3)),
            as.numeric(auc(roc_boot)),
            if (auc(roc_boot) > auc(roc_logit3)) "= Boot" else "logit3 ->"))

# ROC curve plot
par(mfrow = c(1, 1))
plot(roc_logit3,
     col = "steelblue", lwd = 2,
     main = "ROC Curves: mylogit3 vs Bootstrap Ensemble",
     xlab = "1 - Specificity (FPR)", ylab = "Sensitivity (TPR)")
lines(roc_boot, col = "tomato", lwd = 2)
abline(a = 0, b = 1, lty = 2, col = "grey60")
legend("bottomright",
     legend = c(
         sprintf("mylogit3      AUC = %.4f", auc(roc_logit3)),
         sprintf("Bootstrap avg AUC = %.4f", auc(roc_boot))),
     col = c("steelblue", "tomato"),
     lwd = 2, bty = "n")

# Calibration: predicted probability histograms
# Shows HOW the two models distribute their predicted probabilities.
# The unbalanced model clusters near ~0.27 (mean of training bad
# rate), so almost nothing exceeds 0.5 => poor sensitivity at t=0.5.
# The bootstrap ensemble spreads predictions around 0.5 correctly.
par(mfrow = c(1, 2))
hist(pred_logit3, breaks = 30, col = "steelblue",
     main = "mylogit3: predicted probs", xlab = "P(default)")
abline(v = 0.5, col = "red", lty = 2)
hist(pred_boot_avg, breaks = 30, col = "tomato",

```

```
main = "Bootstrap avg: predicted probs", xlab = "P(default)")  
abline(v = 0.5, col = "red", lty = 2)
```

3.2

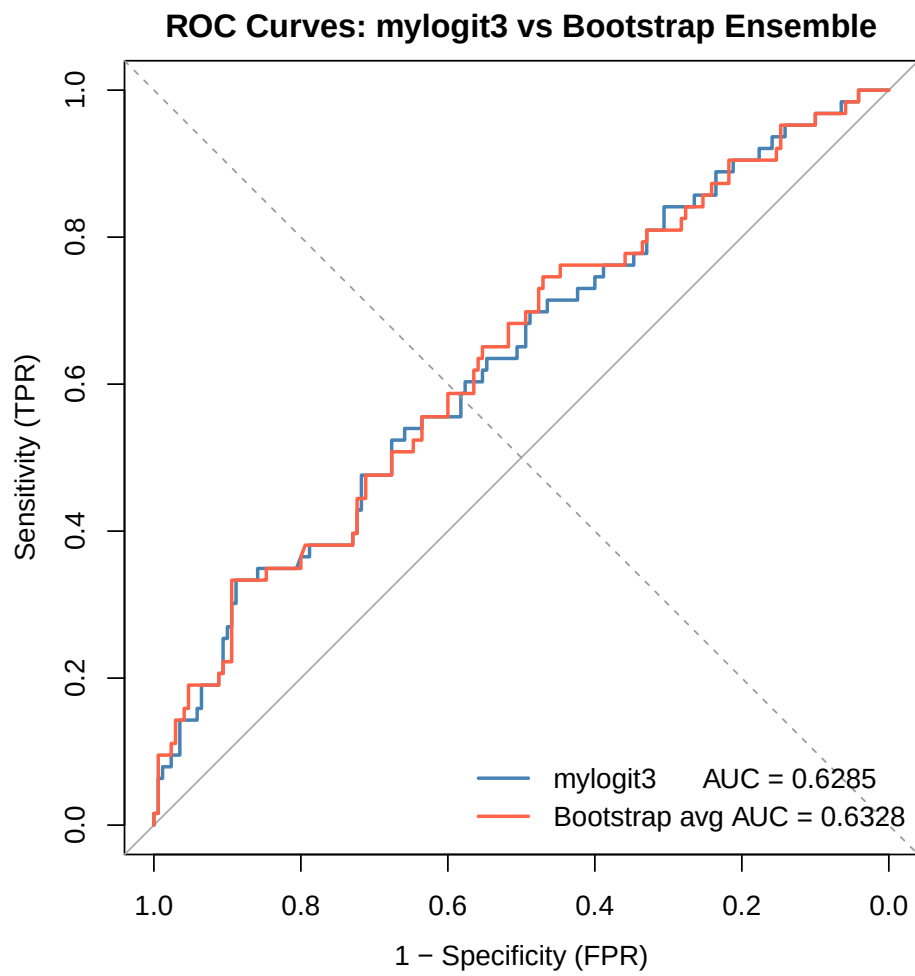


Figure 4: ROC/AUC comparison. Higher value of AUC - better

```
mylogit3                                AUC = 0.6285

Bootstrap ensemble                       AUC = 0.6328

Youden thresholds:  mylogit3=0.351    bootstrap=0.602
```

mylogit3 (t=0.50) Acc=0.738 Sens=0.095 Spec=0.976 F1=0.164 Brier=0.1872
Confusion: TN=166 FP=4 FN=57 TP=6

Bootstrap (t=0.50) Acc=0.627 Sens=0.508 Spec=0.671 F1=0.424 Brier=0.2295
Confusion: TN=114 FP=56 FN=31 TP=32

mylogit3 (t=0.351) Acc=0.738 Sens=0.333 Spec=0.888 F1=0.408 Brier=0.1872
Confusion: TN=151 FP=19 FN=42 TP=21

Bootstrap (t=0.602) Acc=0.742 Sens=0.333 Spec=0.894 F1=0.412 Brier=0.2295
Confusion: TN=152 FP=18 FN=42 TP=21

29